JavaScript Challenges: Higher-Order Functions & Closures

Higher-Order Function Challenges

1. Pass a Function as Argument

Create a function repeatTwice that takes another function and calls it twice.

Example:

```
function greet() {
  console.log("Hello!");
}
repeatTwice(greet); // Output: "Hello!" (2 times)
```

2. Return a Function

Write a function multiplier(factor) that returns a new function which multiplies a number by factor.

Example:

```
const double = multiplier(2);
console.log(double(5)); // 10
```

3. Filter Custom Condition

Create a filterGreaterThan function that takes an array and a limit, and returns only the elements greater than that limit.

Example:

```
filterGreaterThan([1, 5, 8, 2], 4); // [5, 8]
```

4. Map with Custom Operation

Write a function applyOperation(arr, fn) that applies fn to every element in arr.

Example:

```
const nums = [1, 2, 3];
const squared = applyOperation(nums, n => n * n);
console.log(squared); // [1, 4, 9]
```

5. Compose Two Functions

Create a compose function that combines two functions f and g into a single function that returns f(g(x)).

Example:

```
const double = x => x * 2;
const increment = x => x + 1;
const composed = compose(double, increment);
console.log(composed(3)); // 8
```

Closure Challenges

1. Simple Counter

Create a createCounter() function that returns a function which increments and returns a private count.

Example:

```
const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
```

2. Function Factory

Write createGreeter(name) that returns a function which logs "Hello, <name>!".

Example:

```
const greetJohn = createGreeter("John");
greetJohn(); // "Hello, John!"
```

3. Private Bank Account

Write a createAccount(initial) that returns an object with deposit, withdraw, and balance methods that access a private amount.

Example:

```
const account = createAccount(100);
account.deposit(50);
account.withdraw(30);
console.log(account.balance()); // 120
```

4. Memoization Function

Create a memoize(fn) function that caches results of fn.

Example:

```
function slowSquare(n) {
  console.log("Calculating...");
  return n * n;
}
const memoSquare = memoize(slowSquare);
console.log(memoSquare(5)); // "Calculating..." 25
console.log(memoSquare(5)); // 25 (cached)
```

5. Loop Closure Trap Fix

Fix the following code to print numbers 0 to 4:

```
for (var i = 0; i < 5; i++) {
  setTimeout(function () {
    console.log(i);
  }, 1000);
}
```

Hint: Use closure or let.